

Atividade — Aula 10

TechFood: Tela de Cadastro de Pratos

Aluno(a): Letícia Roberta Oliveira Souto

Turma: 3 B - EM

Data: 11/06/2026

Objetivo da Atividade

Partindo do projeto TechFood entregue na Aula 9, construa a tela de cadastro de pratos (cadastro.html, cadastro.js e cadastro.css), seguindo o mesmo padrão de estilização e de estrutura de arquivos já existente no projeto. A tela deve cadastrar um novo prato e fazer com que ele apareça, de forma organizada, na tela principal (cardápio).

Para o envio da imagem do prato, escolha UMA das estratégias apresentadas nos slides da Aula 10 e implemente-a no seu sistema.

O Que Entregar

1. O projeto TechFood com a tela de cadastro funcionando (link do repositório ou pasta compactada).
2. Este documento preenchido, indicando a estratégia escolhida e como você a aplicou.

1. Estratégia de Imagem/Upload Escolhida

A estratégia escolhida para o upload de fotos no projeto foi o base64, com essa abordagem o arquivo de imagem selecionado pelo usuário é convertido em base64 no próprio frontend antes do envio, assim quando ele chega no back é feita a validação do formato da imagem, o helper vai reconstruir essa imagem e implementar o caminho correto da foto, após isso o controller vai tirar a parte do base64 (para deixar o arquivo mais leve), e o service vai salvar no banco, deste modo o front vai receber a confirmação de cadastro concluído.

2. Como Você Implementou no Seu Sistema

No arquivo *cadastro.js*, foi criada uma função para detectar quando o usuário seleciona uma imagem. Nesse momento, o JavaScript utiliza o *FileReader* para converter a imagem em Base64, que é uma forma de representar a imagem como texto. Essa string também é usada para exibir uma pré-visualização da foto na tela.

Quando o usuário cadastra o produto, o Base64 é enviado junto com os demais dados em uma requisição para o backend.

No backend, o Controller recebe os dados e envia o Base64 para o helper. O helper verifica se o conteúdo está em um formato válido, reconstrói a imagem original e a salva na pasta *uploads*. Em seguida, o Controller remove o Base64 e mantém apenas o nome do arquivo para ser salvo no banco de dados.

Por fim, quando o cardápio é carregado, o sistema busca os produtos cadastrados no banco. O frontend utiliza o nome do arquivo salvo para montar o caminho da imagem na pasta *uploads* e exibi-la corretamente nos cards dos produtos.

3. Arquivos Criados ou Alterados

cadastro.html: Arquivo criado do zero contendo a estrutura semântica do formulário de cadastro de pratos (nome, descrição, preço, categoria e seleção de foto) alinhado à identidade visual do TechFood.

cadastro.js: Arquivo criado do zero, responsável por capturar a imagem selecionada pelo usuário, convertê-la para Base64, exibir uma pré-visualização na tela, validar os campos do formulário e enviar os dados do produto para a API.

cadastro.css: Arquivo criado do zero para estilizar o layout do formulário, botões, caixas de input e a janela de preview da imagem.

index.html: Arquivo alterado para incluir o link de navegação “Cadastrar Prato” dentro da tag <nav> no topo da página.

main.js: Arquivo alterado na função de renderização para extrair e validar os dados de produtos em array protegendo contra respostas encapsuladas da API e mapear a propriedade *fotoBase64* no carregamento dos cards dinâmicos.

4. Dificuldades e Como Resolveu

1. Duplicação de Pratos no Cardápio

Dificuldade: Durante os testes iniciais do formulário de cadastro, múltiplos cliques de envio ou testes sucessivos de requisições dispararam várias inserções idênticas no banco de dados. Como o frontend (*main.js*) está programado para renderizar dinamicamente tudo o que a API retorna do MySQL, a tela da vitrine passou a exibir o mesmo item repetidas vezes.

Como Resolvi: Foi necessária uma manutenção direta na camada de dados (banco de dados). Acessei o SGDB, deletei a tabela de produtos para limpar os registros redundantes gerados pelos testes e a recriei limpa, rodando novamente o script SQL de população inicial (*seed*) para garantir a integridade dos dados expostos no cardápio.

2. Falha no salvamento das informações do produto

Dificuldade: A string Base64 da imagem era enviada com sucesso no corpo da requisição, porém o restante das informações essenciais do prato (nome, descrição, preço e categoria) não estava sendo registrado no banco de dados. Isso ocorreu devido a uma quebra de sincronia entre os nomes das chaves (propriedades) enviadas pelo JSON do frontend no arquivo *cadastro.js* e as variáveis esperadas pelo backend no *req.body*.

Como Resolvi: A solução foi analisar o arquivo controlador do backend para mapear e alinhar exatamente o nome de cada propriedade enviada pelo objeto JavaScript (ex: nome, preco,

categoria, descricao) com o que o backend precisava desestruturar. Também garanti no frontend a correta conversão de tipos (garantindo que o preço fosse enviado como número decimal via parseFloat e não como texto puro), o que destravou a validação do servidor.

3. Erro no Fetch

Esse erro deu na última aula antes de eu enviar o documento (18/06) e não consegui resolvê-lo.):

Checklist Final (autoavaliação)

- ☒ ~~Criei cadastro.html, cadastro.js e cadastro.css seguindo o padrão do projeto.~~
- ☒ ~~O formulário valida os campos antes de enviar.~~
- ☐ O cadastro envia os dados ao back-end (POST /produtos) com sucesso.
- ☐ O prato cadastrado aparece na tela principal de forma organizada.
- ☒ ~~Preenchi e anexe este documento.~~